

Chapter 5. Fault Tolerance and Catastrophe-Preparedness

A production-ready microservice is fault tolerant and prepared for any catastrophe. Microservices will fail, they will fail often, and any potential failure scenario can and will happen at some point within the microservice's lifetime. Ensuring availability across the microservice ecosystem requires careful failure planning, preparation for catastrophes, and actively pushing the microservice to fail in real time to ensure that it can recover from failures gracefully.

This chapter covers avoiding single points of failure, common catastrophes and failure scenarios, handling failure detection and remediation, implementing different types of resiliency testing, and ways to handle incidents and outages at the organizational level when failures do occur.

Principles of Building Fault-Tolerant Microservices

The reality of building large-scale distributed systems is that individual components can fail, they will fail, and they will fail often. No microservice ecosystem is an exception to this rule. Any possible failure scenario can and will happen at some point in a microservice's lifetime, and these failures are made worse by the complex dependency chains within microservice ecosystems: if one service in the dependency chain fails, all of the upstream clients will suffer, and the end-to-end availability of the entire system will be compromised.

The only way to mitigate catastrophic failures and avoid compromising the availability of the entire system is to require each microservice within the ecosystem to be *fault tolerant* and *prepared for any catastrophe*.

The first step involved in building a fault-tolerant, catastrophe-prepared microservice is to architect away *single points of failure*. There should never be one piece of the ecosystem whose failure can bring the entire system to a halt, nor should there be any individual piece within the ar-

chitecture of a microservice that will bring the microservice down whenever it fails. Identifying these single points of failure, both within the microservice and at a layer of abstraction above it, can prevent the most glaring failures from occurring.

Identifying failure scenarios is the next step. Not every failure or catastrophe that befalls a microservice is a glaringly obvious single point of failure that can be architected away. Fault tolerance and catastrophe-preparedness require that a microservice withstand both *internal failures* (failures within the microservice itself) and *external failures* (failures within other layers of the ecosystem). From a host failure to the failure of an entire datacenter, from a database to a service's distributed task queue, the number of ways in which a microservice can be brought down by the failure of one or more of its parts is overwhelming, scaling with the complexity of both the microservice itself and the microservice ecosystem as a whole.

Once single points of failure have been architected away and most (if not all) failure scenarios have been identified, the next step is to test for these failures to see whether or not the microservice can recover gracefully when these failures occur, and determine whether or not it is resilient. The resiliency of a service can be tested very thoroughly through *code testing*, *load testing*, and *chaos testing*.

This step is crucial: in a complex microservice ecosystem, merely architecting away failure is not enough—even the best mitigation strategy can turn out to be completely useless when components begin to fail. The only way to build a truly fault-tolerant microservice is to push it to fail in production by actively, repeatedly, and randomly failing each component that could cause the system to break.

Not all failures can be predicted, so the last steps in building fault-tolerant, catastrophe-prepared microservices are organizational in nature. Failure detection and mitigation strategies need to be in place and should be standardized across each microservice team, and every new failure that a service experiences should be added to the resiliency testing suite to ensure it never happens again. Microservice teams also need to be trained to handle failures appropriately: dealing with outages and incidents (regardless of severity) should be standardized across the engineering organization.

- It has no single point of failure.
 - All failure scenarios and possible catastrophes have been identified.
 - It is tested for resiliency through code testing, load testing, and chaos testing.
 - Failure detection and remediation has been automated.
 - There are standardized incident and outage procedures in place within the microservice development team and across the organization.
-

Avoiding Single Points of Failure

The first place to look for possible failure scenarios is within the architecture of each microservice. If there is one piece of the service's architecture that would bring down the entire microservice if it were to fail, we refer to it as a *single point of failure* for the microservice. No one piece of a microservice's architecture should be able to bring down the service, but they frequently do. In fact, most microservices in the real world don't have just one *single* point of failure but have *multiple* points of failure.

EXAMPLE: MESSAGE BROKER AS A SINGLE POINT OF FAILURE

To understand what a single point of failure would look like in the real production world, let's consider a microservice written in Python that uses a combination of Redis (as message broker) and Celery (as task processor) for distributed task processing.

Let's say that the Celery workers (which are processing the tasks) break down for some reason and are unable to complete any of their work. This isn't necessarily a point of failure, because Redis (acting as the message broker) can retry the tasks when the workers are repaired. While the workers are down, Redis stays up, and the tasks build up in the queue on Redis, waiting to be distributed to the Celery workers once they are back up and running. This microservice, however, hosts a *lot* of traffic (receiving thousands of requests per second), and the queues begin to back up, filling up the entire capacity of the Redis machine. Before you know it, the Redis box is out of memory, and you start losing tasks. This sounds bad enough, but the situation can become even worse than it might at first appear, because your hardware might be shared between many different microservices, and now every other microservice that is using this Redis box as a message broker is losing all of their tasks.

This (the Redis machine in this example) is a single point of failure, and it's a real-world example I've seen many, many time in my experience of working with developers to identify single points of failure in their microservices.

It's easy to identify points of failure within microservices when they actually fail, and we need to fix them in order to bring the microservice back up. Waiting for the failure, however, isn't the best approach if we want our microservices to be fault tolerant and preserve their availability. A great way to discover points of failure before they are responsible for an outage is to run architecture reviews with microservice development teams, ask the developers on each team to draw the architecture of their microservice on a whiteboard, and then walk them through the architecture, asking, "What happens if *this* piece of the microservice architecture fails?" (see "[Microservice Understanding](#)" for more details on architecture reviews and discovering single points of failure).

Due to the complex dependency chains that exist between different microservices within a microservice ecosystem, a point of failure in the architecture of one individual microservice is often a point of failure for the entire dependency chain, and in extreme cases, for the entire ecosystem. There are no isolated points of failure within microservice ecosystems, which makes identifying, mitigating, and architecting away points of failure essential for achieving fault-tolerance.

Once any single (or multiple) points of failure have been identified, they need to be mitigated, and (if possible) architected away. If the point of failure can be completely architected away and replaced by something more fault tolerant, then the problem is solved. Sadly, we can't always avoid every single way in which a service can fail, and there are some situations in which we can't architect away even the most glaringly obvious points of failure. For example, if our engineering organization mandates the use of a certain technology that works well for the rest of the development teams but represents a single point of failure for our service, then there may not be a way to architect it away, and our only option for bringing our service toward a fault-tolerant state is to find ways of mitigating any negative consequences of its failure.

Catastrophes and Failure Scenarios

If we know anything about complex systems and large-scale distributed system architecture, it's this: that the system will break in any way that it can be broken, and any failure that could possibly happen will almost assuredly happen at some point during the system's lifetime.

Microservices are complex systems. They are part of large-scale distributed systems (microservice ecosystems) and are therefore no exception to this rule. Any possible failure and any possible catastrophe will almost assuredly happen at some point in between the time a microservice's request for comments (RFC) is written up and the time the microservice is being deprecated and decommissioned. Catastrophes happen all of the time: racks fail in datacenters, HVAC systems break, production databases are deleted by accident (yes, this happens more than most developers would like to admit), natural disasters wipe out entire datacenters. Any failure that can happen will happen: dependencies will fail,

individual servers will fail, libraries will become corrupted or lost entirely, monitoring will fail, logs can and will be lost (seemingly vanishing into thin air).

Once we've identified, mitigated, and (if possible) architected away any glaringly obvious points of failures in our microservice's architecture, the next step is to *identify any other failure scenarios and potential catastrophes* that could befall our microservice. We can separate these types of failures and catastrophes into four main categories, organizing them using their place in the microservice ecosystem stack. The most common catastrophes and failure scenarios are hardware failures, infrastructure (communication-layer and application-platform-layer) failures, dependency failures, and internal failures. We'll look closely at some of the most common possible failure scenarios within each of these categories in the following sections, but first we'll cover a few common causes of failures that affect every level of the microservice ecosystem.

I should note that the lists of possible failure scenarios presented here are not exhaustive. The objective here is to present the most common scenarios and encourage the reader to determine what sorts of failures and catastrophes their microservice(s) and microservice ecosystem(s) may be susceptible to, and then (where necessary) refer the reader to other chapters within this book where some of the relevant topics are covered. Most of the failures here can be avoided by adopting the production-readiness standards (and implementing their corresponding requirements) found throughout this book, so I've only mentioned a few of the failures, and haven't included every failure that's covered in the other chapters.

Common Failures Across an Ecosystem

There are some failures that happen at every level of the microservice ecosystem. These sorts of failures are usually caused (in some way or other) by the lack of standardization across an engineering organization, because they tend to be operational (and not necessarily technical) in nature. Referring to them as “operational” doesn't mean that they are less important or less dangerous than technical failures, nor does it mean that resolving these failures isn't within the technical realm and isn't the responsibility of microservice development teams. These types of failures tend to be the most serious, have some of the most debilitating technical consequences, and reflect a lack of alignment across the various engi-

neering teams within an organization. Of these types of failures, the most common are insufficient design reviews of system and service architecture, incomplete code reviews, poor development processes, and unstable deployment procedures.

Insufficient design reviews of system and microservice architecture lead to poorly designed services, especially within large and complex microservice ecosystems. The reason for this is simple: no one engineer and no one microservice development team will know the details of the infrastructure and the complexity of all four levels of the ecosystem. When new systems are being designed, and new microservices are being architected, it's vital to the future fault tolerance of the system or service that engineers from each level of the microservice ecosystem are brought into the design process to determine how the system or service should be built and run given the intricacies of the entire ecosystem. However, even if this is done properly when the system or service is first being designed, microservice ecosystems evolve so quickly that the infrastructure is often practically unrecognizable after a year or two, and so scheduled reviews of the architecture with experts from each part of the organization can help to ensure that the system or microservice is up-to-date and fits into the overall ecosystem appropriately. For more details on architecture reviews, see [*Chapter 7, Documentation and Understanding*](#).

Incomplete code reviews are another common source of failure. Even though this problem is not specific to microservice architecture, the adoption of microservice architecture tends to exacerbate the problem. Given the higher developer velocity that comes along with microservices, developers are often required to review any new code written by their teammates several times each day in addition to writing their own code, attending meetings, and doing everything else that they need to accomplish to run their service(s). This requires constant context-switching, and it's easy to lose attention to details within someone else's code when you barely have enough time to review your own before deploying it. This leads to countless bugs being introduced into production, bugs that cause services and systems to fail, bugs that could have been caught with better code review. There are several ways to mitigate this, but it can't ever be completely resolved in an environment with high developer velocity. Care needs to be taken to write extensive tests for each system and service, to test each new change extensively before it hits production, and to ensure that, if bugs aren't caught before they are deployed, they're caught else-

where in the development process or in the deployment pipeline, which leads us to our next two common causes of failure.

One of the leading causes of outages in microservice ecosystems are bad deployments. “Bad” deployments are those that contain bugs in the code, broken builds, etc. Poor development processes and unstable deployment procedures allow failures to be introduced into production, bringing down any system or service that the failure-inducing problem is deployed to along with any (and sometimes all) of its dependencies. Putting good code review procedures into place, and creating an engineering culture where both code review is taken seriously and developers are given adequate time to focus on reviewing their teammates’ code is the first step toward avoiding these kinds of failures, but many of them will still go uncaught: even the best code reviewers can’t predict exactly how a code change or new feature will behave in production without further testing. The only way to catch these failures before they bring the system or service down is to build stable and reliable development processes and deployment pipelines. The details of building stable and reliable development processes and deployment pipelines are covered in [Chapter 3, *Stability and Reliability*](#).

SUMMARY: COMMON FAILURES ACROSS AN ECOSYSTEM

The most common failures that happen across all levels of microservice ecosystems are:

- Insufficient design reviews of system and service architecture
 - Incomplete code reviews
 - Poor development processes
 - Unstable deployment procedures
-

Hardware Failures

The lowest layer of the stack is where the hardware lies. The hardware layer is comprised of the actual, physical computers that all of the infrastructure and application code run on, in addition to the racks the servers are stored in, the datacenters where the servers are running, and in the case of cloud providers, regions and availability zones. The hardware layer also contains the operating system, resource isolation and ab-

straction, configuration management, host-level monitoring, and host-level logging. (For more details about the hardware layer of the microservice ecosystems, turn to [Chapter 1, *Microservices*](#).)

Much can go wrong within this layer of the ecosystem, and it is the layer that genuine catastrophes (and not just failures) affect the most. It's also the most delicate layer of the ecosystem: if the hardware fails and there aren't alternatives, the entire engineering organization goes down with it. The catastrophes that happen here are pure hardware failures: a machine dies or fails in some way, a rack goes down, or an entire datacenter fails. These catastrophes happen more often than we would like to admit, and in order for a microservice ecosystem to be fault tolerant, in order for any individual microservice to be fault tolerant and prepared for these catastrophes, these failures need to be planned for, mitigated, and protected against.

Everything else within this layer that lies on top of the bare machines can fail, too. Machines need to be provisioned before anything can run on them, and if provisioning fails, then utilizing any new machines (or, in some cases, even old machines) won't be able to happen. Many microservice ecosystems that utilize technologies that support resource isolation (like Docker) or resource abstraction and allocation (like Mesos and Aurora) can also break or fail, and their failures can bring the entire ecosystem to a halt. Failures caused by broken configuration management or configuration changes are extraordinarily common as well, and are often difficult to detect. Monitoring and logging can fail miserably here as well, and if host-level monitoring and logging fails in some way, triaging any outages becomes impossible because the data needed to mitigate any problems won't be available. Network failures (both internal and external) can also happen. Finally, operational downtimes of critical hardware components—even if communicated properly throughout the organization—can lead to outages across the ecosystem.

Some of the most common hardware failure scenarios are:

- Host failure
 - Rack failure
 - Datacenter failure
 - Cloud provider failure
 - Server provisioning failure
 - Resource isolation and/or abstraction technology failure
 - Broken configuration management
 - Failures caused by configuration changes
 - Failures and gaps in host-level monitoring
 - Failures and gaps in host-level logging
 - Network failure
 - Operational downtimes
 - Lack of infrastructure redundancy
-

Communication-Level and Application Platform-Level Failures

The second and third layers of the microservice ecosystem stack are comprised of the communication and application platform layers. These layers live between the hardware and the microservices, bridging the two as the glue that holds the ecosystem together. The communication layer contains the network, DNS, the RPC framework, endpoints, messaging, service discovery, service registry, and load balancing. The application platform layer is comprised of the self-service development tools, development environment, test and package and release and build tools, the deployment pipeline, microservice-level logging, and microservice-level monitoring—all critical to the day-to-day running and building of the microservice ecosystem. Like hardware failures, failures that happen at these levels compromise the entire company, because every aspect of development and maintenance within the microservice ecosystem depends critically on these systems running smoothly and without failure. Let's take a look at some of the most common failures that can happen within these layers.

Within the communication layer, network failures are especially common. These can be failures of the internal network(s) that all remote procedure calls are made over, or failures of external networks. Another type of network-related failure arises from problems with firewalls and improper iptables entries. DNS errors are also quite common: when DNS errors happen, communication can grind to a halt, and DNS bugs can be rather difficult to track down and diagnose. The RPC layer of communication—the glue that holds the entire delicate microservice ecosystem together—is another (rather infamous) source of failure, especially when there is only one channel connecting all microservices and internal systems; setting up separate channels for RPC and health checks can mitigate this problem a bit if health checks and other related monitoring is kept separate from channels that handle data being passed between services. It's possible for messaging systems to break (as I mentioned briefly in the Redis-Celery example earlier in this chapter), and messaging queues, message brokers, and task processors often live in microservice ecosystems without any backups or alternatives, acting as frightening points of failure for every service that relies on them. Failures of service discovery, service registry, and load balancing can (and do) happen as well: if any part of these systems breaks or experiences downtime without any alternatives, then traffic won't be routed, allocated, and distributed properly.

Failures within the application platform are more specific to the way that engineering organizations have set up their development process and deployment pipeline, but as a rule, these systems can fail just as often and as catastrophically as every other service within the ecosystem stack. If development tools and/or environments are working incorrectly when developers are trying to build new features or repair existing bugs, bugs and new failure modes can be introduced into production. The same goes for any failures or shortcomings of the test, package, build, and release pipelines: if packages and builds contain bugs or aren't properly put together, then deployments will fail. If the deployment pipeline is unavailable, buggy, or fails outright, then deployment will grind to a halt, preventing not only deployment of new features but of critical bug-fixes that may be in the works. Finally, monitoring and logging of individual microservices can contain gaps or fail as well, making triaging or logging any issues impossible.

Some of the most common communication and application platform failures are:

- Network failures
 - DNS errors
 - RPC failures
 - Improper handling of requests and/or responses
 - Messaging system failures
 - Failures in service discovery and service registry
 - Improper load balancing
 - Failure of development tools and development environment
 - Failures in the test, package, build, and release pipelines
 - Deployment pipeline failures
 - Failures and gaps in microservice-level logging
 - Failures and gaps in microservice-level monitoring
-

Dependency Failures

Failures within the top level of the microservice ecosystem (the microservice layer) can be divided into two separate categories: (1) those that are internal to a specific microservice and caused by problems within it, and (2) those that are external to a microservice and caused by the microservice's dependencies. We'll cover common failure scenarios within this second category first.

Failures and outages of a downstream microservice (that is, one of a microservice's dependencies) are extraordinarily common and can dramatically affect a microservice's availability. If even one microservice in the dependency chain goes down, it can take all of its upstream clients down with it if there are no protections in place. However, a microservice doesn't always necessarily need to experience a full-blown outage in order to negatively affect the availability of its upstream clients—if it fails to meet its SLA by just one or two nines, the availability of all upstream client microservices will drop.

Microservices can cause their upstream clients to fail to meet their SLAs. If a service's availability drops by one or two nines, all upstream clients suffer, all thanks to how the math works: the availability of a microservice is calculated as its own availability multiplied by the availability of its downstream dependencies. Failing to meet an SLA is an important (and often overlooked) microservice failure, and it's a failure that brings down the availability of every other service that depends on it (along with the services that depend on those services).

Other common dependency failures are those caused by timeouts to another service, the deprecation or decommissioning of a dependency's API endpoints (without proper communication regarding the deprecation or decommissioning to all upstream clients), and the deprecation or decommissioning of an entire microservice. In addition, versioning of internal libraries and/or microservices and pinning to specific versions of internal libraries and/or services is very much discouraged in microservice architecture because it tends to lead to bugs and (in extreme cases) serious failures, because of the fast-paced nature of microservice development: these libraries and services are constantly changing, and pinning to specific versions (along with versioning of these services and libraries in general) can lead to developers using unstable, unreliable, and sometimes unsafe versions of them.

Failures of external dependencies (third-party services and/or libraries) can and do happen as well. These can be more difficult to detect and fix than failures of internal dependencies, because developers will have little to no control over them. The complexity associated with depending on third-party services and/or libraries can be handled properly if these scenarios are anticipated from the beginning of the microservice's lifecycle: choose established and stable external dependencies, and try to avoid using them unless completely necessary, lest they become a single point of failure for your service.

Some of the most common dependency failure scenarios are:

- Failures or outages of a downstream (dependency) microservice
 - Internal service outages
 - External (third-party) service outages
 - Internal library failures
 - External (third-party) library failures
 - A dependency failing to meet its SLA
 - API endpoint deprecation
 - API endpoint decommissioning
 - Microservice deprecation
 - Microservice decommissioning
 - Interface or endpoint deprecation
 - Timeouts to a downstream service
 - Timeouts to an external dependency
-

Internal (Microservice) Failures

At the very top of the microservice ecosystem stack lie the individual microservices. To the development teams, these are the failures that matter the most, because they are completely dependent on good development practices, good deployment practices, and the ways in which development teams architect, run, and maintain their individual microservices.

Assuming that the infrastructure below the microservice layer is relatively stable, the majority of incidents and outages experienced by a microservice will be almost solely self-inflicted. Developers on call for their services will find themselves paged almost solely for issues and failures whose root causes are found within their microservice—that is, the alerts they will receive will have been triggered by changes in their microservice’s key metrics (see [Chapter 6, Monitoring](#), for more information about monitoring, logging, alerting, and microservice key metrics).

Incomplete code reviews, lack of proper test coverage, and poor development processes in general (specifically, the lack of a standardized development cycle) lead to buggy code being deployed to production—failures that can be avoided by standardizing the development process across mi-

microservice teams (see [“The Development Cycle”](#)). Without a stable and reliable deployment pipeline containing staging, canary, and production phases in place to catch any errors before they are fully rolled out to production servers, any problems not caught by testing in the development phases can cause serious incidents and outages for the microservice itself, its dependencies, and any other parts of the microservice ecosystem that depend on it.

Anything specific to the microservice’s architecture can also fail here, including any databases, message brokers, task-processing systems, and the like. This is also where general and specific code bugs within the microservice will cause failures, as well as improper error and exception handling: unhandled exceptions and the practice of catching exceptions are an often-overlooked culprit when microservices fail. Finally, increases in traffic can cause a service to fail if the service isn’t prepared for unexpected growth (for more on scalability limitations, turn to [Chapter 4, *Scalability and Performance*](#), and then read [“Load Testing”](#) of the current chapter).

SUMMARY: COMMON MICROSERVICE FAILURE SCENARIOS

Some of the most common microservice failures are:

- Incomplete code reviews
- Poor architecture and design
- Lack of proper unit and integration tests
- Bad deployments
- Lack of proper monitoring
- Improper error and exception handling
- Database failure
- Scalability limitations

Resiliency Testing

Architecting away single points of failure and identifying possible failure scenarios and catastrophes isn’t enough to ensure that microservices are fault tolerant and prepared for any catastrophe. In order to be truly fault tolerant, a microservice must be able to experience failures and recover

from them gracefully without affecting their own availability, the availability of their clients, and the availability of the overall microservice ecosystem. The single best way to ensure that a microservice is fault tolerant is to take all of the possible failure scenarios that it could be affected by, and then actively, repeatedly, and randomly push it to fail in production—a practice known as *resiliency testing*.

A resilient microservice is one that can experience and recover from failures at every level of the microservice ecosystem: the hardware layer (e.g., a host or datacenter failure), the communication layer (e.g., RPC failures), the application layer (e.g., a failure in the deployment pipeline), and in the microservice layer (e.g., failure of a dependency, a bad deployment, or a sudden increase in traffic). There are several types of resiliency testing that, when used to evaluate the fault tolerance of a microservice, can ensure that the service is prepared for any known failures within any layer of the stack.

The first type of resiliency testing we will look at is *code testing*, which is comprised of four types of tests that check syntax, style, individual components of the microservice, how the components work together, and how the microservice performs within its complex dependency chains. (Code testing usually isn't considered part of the resiliency testing suite, but I wanted to include it here for two reasons: first, since it is crucial for fault tolerance and catastrophe-preparedness, it makes sense to keep it in this section; second, I've noticed that development teams have preferred to keep all testing information in one place.) The second is *load testing*, in which microservices are exposed to higher traffic loads to see how they behave under increased traffic. The third type of resiliency testing is *chaos testing*, which is the most important type of resiliency testing, in which microservices are actively pushed to fail in production.

Code Testing

The first type of resiliency testing is *code testing*, a practice almost all developers and operational engineers are familiar with. In microservice architecture, code testing needs to be run at every layer of the ecosystem, both within the microservices and on any system or service that lives in the layers below: in addition to microservices, service discovery, configuration management, and related systems also need to have proper code

testing in place. There are several types of good code testing practices, including *lint testing*, *unit testing*, *integration testing*, and *end-to-end testing*.

Lint tests

Syntax and style errors are caught using *lint testing*. Lint tests run over the code, catching any language-specific problems, and also can be written to ensure that code matches language-specific (and sometimes team-specific or organization-specific) style guidelines.

Unit tests

The majority of code testing is done through *unit tests*, which are small and independent tests that are run over various small pieces (or units) of the microservice's code. The goal of unit tests is to make sure that the software components of the service itself (e.g., functions, classes, and methods) are resilient and don't contain any bugs. Unfortunately, many developers only consider unit tests when writing tests for their applications or microservices. While unit testing is good, it's not good enough to evaluate the actual ways in which the microservice will behave in production.

Integration tests

While unit tests evaluate small pieces of the microservice to ensure that the components are resilient, the next type of code tests are *integration tests*, which test how the entire service works. In integration testing, all of the smaller components of the microservice (which were testing individually using unit tests) are combined and tested together to make sure that they work as expected when they need to work together.

End-to-end tests

For a monolithic or standalone application, often unit tests and integration tests are good enough together to comprise the code testing aspect of resiliency testing, but microservice architecture introduces a new level of complexity within code testing due to the complex dependency chains that exist between a microservice, its clients, and its dependencies. Another additional set of tests need to be added to the code testing suite that evaluate the behavior of the microservice with respect to its clients and dependencies. This means that microservice developers need to build

end-to-end tests that run just like real production traffic, tests that hit the endpoints of their microservice's clients, hit their own microservice's endpoints, hit the endpoints of the microservice's dependencies, send read requests to any databases, and catch any problems in the request flow that might have been introduced with a code change.

Automating code tests

All four types of code tests (lint, unit, integration, and end-to-end) should be written by the development team, but running them should be automated as part of the development cycle and the deployment pipeline. Unit and integration tests should run during the development cycle on an external build system, right after changes have made it through the code review process. If the new code changes fail any unit or integration tests, then they should not be introduced into the deployment pipeline as a candidate for production, but should be rejected and brought to the attention of the development team for repair. If the new code changes pass all unit and integration tests, then the new build should be sent to the deployment pipeline as a candidate for production.

SUMMARY OF CODE TESTING

The four types of production-ready code testing are:

- Lint tests
 - Unit tests
 - Integration tests
 - End-to-end tests
-

Load Testing

As we saw in [Chapter 4, Scalability and Performance](#), a production-ready microservice needs to be both scalable and performant. It needs to handle a large number of tasks or requests at the same time and handle them efficiently, and it also must be prepared for tasks or requests to increase in the future. Microservices that are unprepared for increases in traffic, tasks, or requests can experience severe outages when any of these gradually or suddenly increase.

From the point of view of a microservice development team, we know that traffic to our microservice will mostly likely increase at some time in the future, and we might even know by exactly how much the traffic will increase. We want to be fully prepared for these increases in traffic so that we can avoid any potential problems and/or failures. In addition, we want to illuminate any possible scalability challenges and bottlenecks that we might not be aware of until our microservice is pushed to the very limits of its scalability. To protect against any scalability-related incidents and outages, and to be fully prepared for future increases in traffic, we can test the scalability of our services using *load testing*.

Fundamentals of load testing

Load testing is exactly what its name implies: it is a way to test how a microservice behaves under a specific traffic load. During load testing, a target traffic load is chosen, the target load of test traffic is run on the microservice, and then the microservice is monitored closely to see how it behaves. If the microservice fails or experiences any issues during load testing, its developers will be able to resolve any scalability issues that appear in load tests that would have otherwise harmed the availability of their microservice in the future.

Load testing is where the growth scales and resource bottlenecks and requirements that were covered in [Chapter 4, *Scalability and Performance*](#), come in handy. From a microservice's qualitative growth scale and the associated high-level business metrics, development teams can learn how much traffic their microservice should be prepared to handle in the future. From the quantitative growth scale, developers will know exactly how many requests or queries per second their service will be expected to handle. If the majority of the service's resource bottlenecks and resource requirements have been identified, and the bottlenecks architected away, developers will know how to translate the quantitative growth scale (and, consequently, the quantitative aspects of future increases in traffic) into terms of the hardware resources their microservice will require in order to handle higher traffic loads. Load testing after all of this, after applying the scalability requirements and working through them, can validate and ensure that the microservice is ready for the expected increase in traffic.

Load testing can be used the other way around, to discover the quantitative and qualitative growth scales, to identify resource bottlenecks and requirements, to ensure dependency scaling, to determine and plan for future capacity needs, and the like. When done well, load testing can give developers deep insight into the scalability (and scalability limitations) of their microservice: it measures how the service, its dependencies, and the ecosystem behave in a controlled environment under a specified traffic load.

Running load tests in staging and production

Load testing is most effective when it is run on each stage of the deployment pipeline. To test the load testing framework itself, to make sure that the test traffic produces the desired results, and to catch any potential problems that load testing might cause in production, load testing can be run in the staging phase of the deployment pipeline. If the deployment pipeline is utilizing partial staging, where the staging environment communicates with production services, care needs to be taken to make sure that any load tests run in staging do not harm or compromise the availability of any production services that it communicates with. If the deployment pipeline contains full staging, which is a complete mirror copy of production and where no staging services communicate with any services in production, then care needs to be taken to make sure that load testing in full staging produces accurate results, especially if there isn't host parity between staging and production.

It's not enough to load test only in staging. Even the best staging environments—those that are complete mirror copies of production and have full host parity—still are not production. They're not the real world, and very rarely are staging environments perfectly indicative of the consequences of load testing in production. Once you know the traffic load you need to hit, you've alerted all of the on-call rotations of the dependency teams, and you've tested your load tests in staging, you absolutely need to run load tests in production.

If your load tests send requests to other production services, be sure to alert all dependencies in order to avoid compromising their availability while running load tests. Never assume that downstream dependencies can handle the traffic load you are about to send their way!

Load testing in production can be dangerous and can easily cause a microservice and its dependencies to fail. The reason why load testing is dangerous is the same reason it is essential: most of the time, you won't know exactly how the service being tested behaves under the target traffic load, and you won't know how its dependencies handle increased requests. Load testing is *the* way to explore the unknowns about a service and make sure that it is prepared for expected traffic growth. When a service is pushed to its limits in production, and things begin to break, there need to be automated pieces in place to make sure that any load tests can be quickly shut down. After the limitations of the service have been discovered and mitigated and the fixes have been tested and deployed, load testing can resume.

Automating load testing

If load testing is going to be required for all microservices within the organization (or even just a small number of business-critical microservices), leaving the implementation and methodology of the load testing in the hands of development teams to design and run for themselves introduces another point of failure into the system. Ideally, a self-service load-testing tool and/or system should be part of the application platform layer of the microservice ecosystem, allowing developers to use a trusted, shared, automated, and centralized service.

Load testing should be scheduled regularly, and viewed as an integral component of the day-to-day function of the engineering organization. The scheduling should be linked to traffic patterns: test desired traffic loads in production when traffic is low, and never during peak hours, to avoid compromising the availability of any services. If a centralized self-service load testing system is being used, it is incredibly useful to have an automated process for validating new tests, along with a suite of trusted (and required) tests that every service can run. In extreme cases, and

when a self-service load testing tool is reliable, deployments can be blocked (or gated) if a microservice fails to perform adequately under load tests. Most importantly, every load test performed needs to be sufficiently logged and publicized internally so that any problems caused by load testing can quickly be detected, mitigated, and resolved.

SUMMARY OF LOAD TESTING

Production-ready load testing has the following components:

- It uses a target traffic load that is calculated using the qualitative and quantitative growth scales and expressed in terms of RPS, QPS, or TPS.
- It is run in each stage of the deployment pipeline.
- Its runs are communicated to all dependencies.
- It is fully automated, is logged, and is scheduled.

Chaos Testing

In this chapter, we've seen various potential failure scenarios and catastrophes that can happen at each layer of the stack. We've seen how code testing catches small potential failures at the individual microservice level, and how load testing catches failures that arise from scalability limitations at the microservice level. However, the majority of the failure scenarios and potential catastrophes lie elsewhere in the ecosystem and cannot be caught by any of these kinds of tests. To test for *all* failure scenarios, to make sure that microservices can gracefully recover from any potential catastrophe, there's one additional type of resiliency testing that needs to be in place, and it's known (quite appropriately) as *chaos testing*.

In chaos testing, microservices are actively *pushed to fail* in production, because the only way to make sure that a microservice can survive a failure is to make it fail all of the time, and in every way possible. That means that every failure scenario and potential catastrophe needs to be identified, and then it needs to be forced to happen in production. Running scheduled and random tests of each failure scenario and potential catastrophe can help mimic the real world of complex system failures: developers will know that part of the system will be pushed to fail on a scheduled basis and will prepare for those scheduled chaos runs, and they'll also be caught off guard by randomly scheduled tests.

Chaos testing must be well controlled in order to avoid chaos tests from bringing down the ecosystem. Make sure your chaos testing software has appropriate permissions, and that every single event is logged, so that if microservices are unable to gracefully recover (or if the chaos testing goes rogue), pinpointing and resolving the problems won't require any serious sleuthing.

Like load testing (and many of the other systems covered in this book), chaos testing is best provided as a service, and not implemented in various ad hoc manners across development teams. Automate the testing, require every microservice to run a suite of both general and service-specific tests, encourage development teams to discover additional ways their service can fail, and then give them the resources to design new chaos tests that push their microservices to fail in these new ways. Make sure that every part of the ecosystem (including the chaos testing service) can survive a standard set of chaos tests, and break each microservice and piece of the infrastructure multiple times, again and again and again, until every development and infrastructure team is confident that their services and systems can withstand inevitable failures.

Finally, chaos testing is not just for companies hosted on cloud providers, even though they are the most vocal (and common) users. There are very few differences in failure modes of bare-metal versus cloud provider hardware, and anything that is built to run in the cloud can work just as well on bare metal (and vice versa). An open source solution like Simian Army (which comes with a standard suite of chaos tests that can be customized) will work for the majority of companies, but organizations with specific needs can easily build their own.

Some common types of chaos tests:

- Disable the API endpoint of one of a microservice's dependencies.
 - Stop all traffic requests to a dependency.
 - Introduce latency between various parts of the ecosystem to mimic network problems: between clients and dependencies, between microservices and shared databases, between microservices and distributed task-processing systems, etc.
 - Stop all traffic to a datacenter or a region.
 - Take out a host at random by shutting down one machine.
-

Failure Detection and Remediation

In addition to the resiliency testing suite, in which microservices are tested for every known failure and catastrophe, a production-ready microservice needs to have *failure detection and remediation strategies* for when failures do happen. We'll take a look at organizational processes that can be used across the ecosystem to triage, mitigate, and resolve incidents and outages, but first we'll focus on several technical mitigation strategies in this section.

When a failure does happen, the goal of failure detection and remediation always needs to be the following: *reduce the impact to users*. In a microservice ecosystem, the "users" are whoever may be using the service—this could be another microservice (who is a client of the service) or an actual customer of the product (if the service in question is customer-facing). If the failure in question was (or may have been) introduced into production by a new deployment, the single most effective way to reduce the impact to users when something is going wrong is to immediately roll back to the last stable build of the service. Rolling back to the last stable build ensures that the microservice has been returned to a known state, a state that wasn't susceptible to the failures or catastrophes that were introduced with the newest build. The same holds for changes to low-level configurations: treat configs like code, deploy them in various successive releases, and make sure that if a config change causes an outage, the sys-

tem can quickly and effortlessly roll back to the last stable set of configurations.

A second strategy in case of failure is failing over to a stable alternative. If one of a microservice's dependencies is down, this would mean sending requests to a different endpoint (if the endpoint is broken) or a different service (if the entire service is down). If it's not possible to route to another service or endpoint, then there needs to be a way to queue or save the requests and hold them until problems with the dependency have been mitigated. If the problem is relegated to one datacenter, or if a datacenter is experiencing failures, the way to fail over to a stable alternative would be to re-route traffic to another datacenter. Whenever you are faced with various ways to handle failure, and one of those choices is to re-route traffic to another service or datacenter, re-routing the traffic is almost always the smartest choice: routing traffic is easy and immediately reduces the impact to users.

Importantly, the detection aspect of “failure detection and remediation” can only really be accomplished by production-ready monitoring (see [Chapter 6, *Monitoring*](#), for all the nitty-gritty monitoring details). Human beings are horrible at detecting and diagnosing system failures, and introducing engineers into the failure detection process becomes a single point of failure for the overall system. This holds for failure remediation as well: most of the remediation within large microservice ecosystems is done by engineers, all by hand, all in an almost painfully manual way, introducing a new point of failure for the system—but it doesn't have to be that way. To cut out the potential and possibility for human error in failure remediation, all mitigation strategies need to be automated. For example, if a service fails certain healthchecks or its key metrics hit the warning and/or critical thresholds after a deploy, then the system can be designed to automatically roll back to the last stable build. The same goes for traffic routing to another endpoint, microservice, or datacenter: if certain key metrics hit specific thresholds, set up a system that automatically routes the traffic for you. Fault tolerance absolutely requires that the potential and possibility for human error be automated and architected away whenever possible.

Incidents and Outages

Throughout this book, I've emphasized the availability of the microservices and the overall ecosystem as the goal of standardization. Architecting, building, and running microservice architecture that is geared toward high availability can be accomplished through adopting the production-readiness standards and their related requirements, and it's the reason I've introduced and chosen each production-readiness standard. It's not enough, however, for the individual microservices and each layer of the microservice ecosystem stack to be fault tolerant and prepared for any catastrophe. The development teams and the engineering organization(s) responsible for the microservices and the ecosystem they live in need to have the appropriate organizational response procedures in place for handling incidents and outages when they happen.

Every minute that a microservice is down brings down its availability. When part of the microservice or its ecosystem fails, causing an incident or outage to happen, every minute that it is down counts against its availability and causes it to fail to meet its SLA. Failing to meet an SLA, and failing to meet availability goals, incurs a serious cost: at most companies, outages mean a huge financial cost to the business, a cost that is usually easy to quantify and share with development teams within the organization. With this in mind, it's easy to see how the length of the time to detection, the time to mitigation, and the time to resolution of outages can add up very quickly and cost the company money, because they count against a microservice's uptime (and, consequently, its availability).

Appropriate Categorization

Not all microservices are created equal, and categorizing the importance and impact that their failures will have on the business makes it easier to properly triage, mitigate, and resolve incidents and outages. When an ecosystem contains hundreds or even thousands of microservices, there will be dozens or even hundreds of failures per week, even if only 10 percent of the microservices experience failures, that's still over 100 failures in an ecosystem of 1,000 services. While every failure needs to be properly handled by its on-call rotation, not every failure will need to be treated as an all-hands-on-deck emergency.

In order to have a consistent, appropriate, effective, and efficient incident and outage response process across the organization, it is important to do two things. First, it is incredibly helpful to categorize the microservices

themselves with regard to how their failures will affect the ecosystem so that it will be easy to prioritize various incidents and failures (this also helps with problems related to competition for resources—both engineering resources and hardware resources—within the organization). Second, incidents and outages need to be categorized so that the scope and severity of every single failure will be understood across the organization.

Categorizing microservices

To mitigate the challenges of competition for resources, and to ensure proper incident response measures are taken, each microservice within the ecosystem can (and should) be categorized and ranked according to its criticality to the business. Categorization doesn't need to be perfect at first, as a rough categorization rubric will do the job just fine. The key here is to mark microservices that are critical to the business as having the highest priority and impact, and then every other microservice will have a lower rank and priority depending on how close or far it is to the most critical services. Infrastructure layers are always of the highest priority: anything within the hardware, communication, and application platform layers that is used by any of the business-critical microservices should be the highest priority within the ecosystem.

Categorizing incidents and outages

There are two axes that every incident, outage, and failure can be plotted against: the first is the *severity* of the incident, outage, or failure, and the second is its *scope*. Severity is linked to the categorization of the application, microservice, or system in question. If the microservice is business-critical (i.e., if either the business or an essential customer-facing part of the product cannot function without it), then the severity of any failure it experiences should match the service's categorization. Scope, on the other hand, is related to how much of the ecosystem is affected by the failure, and is usually split into three categories: high, medium, and low. An incident whose scope is high is an incident that affects the entire business and/or an external (e.g., user-facing) feature; a medium-scope incident would be one that affected only the service itself, or the service and a few of its clients; a low-scope incident would be one whose negative effects are not noticed by clients, the business, or external customers using the product. In other words, severity should be categorized based on the im-

pact to the business, and scope should be categorized based on whether the incident is *local* or *global*.

Let's go through a few examples to clarify what this looks like in practice. We'll assign four levels of severity to each failure (0–4, where 0 is the most severe incident level and 4 is the least severe), and we'll stick with the high-medium-low levels when determining scope. First, let's look at an example whose severity and scope are very easy to categorize: a complete datacenter failure. If a datacenter goes completely down (for whatever reason), the severity is clearly 0 (it affects the entire business), and the scope is high (again, it affects the entire business). Now let's look at another scenario: imagine we have a microservice that is responsible for a business-critical function in the product, and it goes down for 30 minutes; as a result of its failure, let's imagine that one of its clients suffers, but the rest of the ecosystem remains unaffected. We'd categorize this as severity 0 (because it impacts a business-critical feature) and scope medium (it doesn't affect the whole business, only itself and one client service). Finally, let's consider an internal tool responsible for generating templates for new microservices, and imagine that it goes down for several hours—how would this be categorized? Generating templates for new microservices (and spinning up new microservices) isn't business-critical and doesn't affect any user-facing features, so this wouldn't be a 0 severity problem (it probably wouldn't be a 1 or a 2 either); however, since the service itself is down, we'd probably categorize its severity as a 3, and then its scope as low (since it is the only service affected by its failure).

The Five Stages of Incident Response

When failures happen, it's critical to the availability of the entire system that there are standardized incident response procedures in place. Having a clear set of steps that need to be taken when an incident or outage occurs cuts down on the time to mitigation and the time to resolution, which in turn decreases the downtime experienced by each microservice. Within the industry today, there are typically three standard steps in the process of responding to and resolving an incident: triage, mitigate, and resolve. Adopting microservice architecture, however, and achieving high availability and fault tolerance requires adopting two additional steps in the incident response process: one for coordination, and another for follow-up. Together, these steps give us the five stages of incident response

([Figure 5-1](#)): *assessment, coordination, mitigation, resolution, and follow-up.*

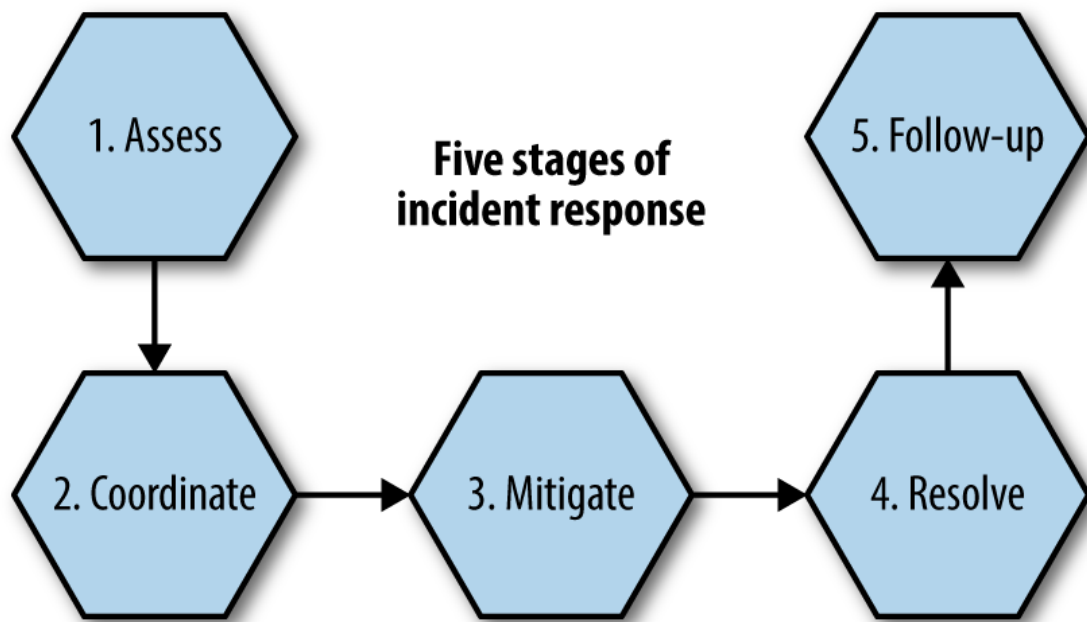


Figure 5-1. The five stages of incident response

Assessment

Whenever an alert is triggered by a change in a service’s key metric (see [Chapter 6, *Monitoring*](#), for more details on alerting, key metrics, and on-call rotations), and the developer on call for the service needs to respond to the alert, the very first step that needs to be taken is to *assess* the incident. The on-call engineer is the first responder, triaging every problem as soon as it triggers an alert, and his job is to determine the severity and scope of the issue.

Coordination

Once the incident has been assessed and triaged, the next step is to first *coordinate* with other developers and teams and then begin *communicating about the incident*. Very few developers on call for any given service will be able to resolve every single problem with the service, and so coordination with other teams who *can* resolve the issue will ensure that the problem is mitigated and resolved quickly. This means that there need to be clear channels of communication for incidents and outages so that any high-severity, high-scope problem can receive the immediate attention that it requires.

During the incident or outage, it's important to have a clear record of communication regarding the incident for several reasons. First, recording communication during the incident (in chat logs, over email, etc.) helps in diagnosing, root-causing, and mitigating the incident: everyone knows who is working on which fix, everyone knows what possible failures have been eliminated as possible causes, and once the root has been identified, everyone knows exactly what caused the problem. Second, other services that depend on the service experiencing the incident or outage need to be apprised of any problems so that they can mitigate its negative effects and ensure that their own service is protected from the failure. This keeps overall availability high, and prevents one service from bringing down entire dependency chains. Third, it helps when post-mortems are written for severe, global incidents by giving a clear, detailed record of exactly what happened and how the problem was triaged, mitigated, and resolved.

Mitigation

The third step is *mitigation*. After the problem has been assessed and organizational communication has begun (ensuring that the right people are working to fix the problem), developers need to work to reduce the impact of the incident on clients, the business, and anything else that may be affected by the incident. Mitigation is not the same as resolution: it is not *fixing* the root cause of the problem completely, only *reducing its impact*. An issue is not mitigated until both its availability and the availability of its clients are no longer compromised or suffering.

Resolution

After the effects of the incident or outage have been mitigated, engineers can work to *resolve* the root cause of the problem. This is the fourth step of the incident response process. This entails actually fixing the root cause of the problem, which may not have been done when the problem was mitigated. Most importantly, this is when the clock stops ticking. The two most important quantities that count against a microservice's SLA are time to detection (TTD) and time to mitigation (TTM). Once a problem has been mitigated, it should no longer be affecting end users or compromising the service's SLA, and so time to resolution (TTR) rarely (if ever) counts against a service's availability.

Follow-up

Three things need to happen in the fifth and final *follow-up* stage of incident response: postmortems need to be written to analyze and understand the incident or outage, severe incidents and outages need to be shared and reviewed, and a list of action items needs to be put together so that the development team(s) can complete them in order for the affected microservice(s) to return to a production-ready state (action items can often be fit into postmortems).

The most important aspect of incident follow-up is the *postmortem*. In general, a postmortem is a detailed document that follows every single incident and/or outage and contains critical information about what happened, why it happened, and what could have been done to prevent it. Every postmortem should, at the very minimum, contain a summary of what happened, data about what happened (time to detection, time to mitigation, time to resolution, total downtime, number of affected users, any relevant graphs and charts, etc.), a detailed timeline, a comprehensive root-cause analysis, a summary of how the incident could have been prevented, ways that similar outages can be prevented in the future, and a list of action items that need to be completed in order to bring the service back to a production-ready state. Postmortems are most effective when they're blameless, when they don't name names but only point out objective facts about the service. Pointing fingers, naming names, and blaming developers and engineers for outages stifles the organizational learning and sharing that is essential for maintaining a reliable, sustainable ecosystem.

Within large and complex microservice ecosystems, any failure or problem that brings one microservice down—whether big or small—almost certainly can (and will) affect at least one other microservice within the ecosystem. Communicating severe incidents and outages across various teams (and across the whole organization) can help catch these failures in other services before they occur. I've seen how effective incidents and outage reviews can be when done properly, and have watched developers attend these meetings and then rush off to their microservice afterward to fix any bugs in their own service that led to the incidents and/or outages that were reviewed.

Evaluate Your Microservice

Now that you have a better understanding of fault tolerance and catastrophe-preparedness, use the following list of questions to assess the production-readiness of your microservice(s) and microservice ecosystem. The questions are organized by topic, and correspond to the sections within this chapter.

Avoiding Single Points of Failure

- Does the microservice have a single point of failure?
- Does it have more than one point of failure?
- Can any points of failure be architected away, or do they need to be mitigated?

Catastrophes and Failure Scenarios

- Have all of the microservice's failure scenarios and possible catastrophes been identified?
- What are common failures across the microservice ecosystem?
- What are the hardware-layer failure scenarios that can affect this microservice?
- What communication-layer and application-layer failures can affect this microservice?
- What sorts of dependency failures can affect this microservice?
- What are the internal failures that could bring down this microservice?

Resiliency Testing

- Does this microservice have appropriate lint, unit, integration, and end-to-end tests?
- Does this microservice undergo regular, scheduled load testing?
- Are all possible failure scenarios implemented and tested using chaos testing?

Failure Detection and Remediation

- Are there standardized processes across the engineering organization(s) for handling incidents and outages?
- How do failures and outages of this microservice impact the business?
- Are there clearly defined levels of failure?
- Are there clearly defined mitigation strategies?
- Does the team follow the five stages of incident response when incidents and outages occur?

[Support](#) [Sign Out](#)